ASSIGNMENT 1 – TASK 9: Introduce Abstraction and Interfaces

Bloom's Level: K4 – Analyse | Marks: 4

For this task, model Vehicle as an interface with default methods, and Chargeable as an abstract class, as required by the alternative concept. This is a deliberate contrast with Task 6's abstract Vehicle class.

```
interface Vehicle {
    String getRegNo();
    double ratePerHour();

    default double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException();
        return ((minutes + 59) / 60) * ratePerHour();
    }

    static boolean validDuration(int minutes) {
        return minutes >= 0;
    }
}

abstract class Chargeable {
    protected final double ratePerUnit;

    Chargeable(double ratePerUnit) {
        this.ratePerUnit = ratePerUnit;
    }

    public double chargingFee(int units) {
        if (units < 0) throw new IllegalArgumentException();
        return units * ratePerUnit;
    }
}
```

In this alternative, Vehicle's essential methods getRegNo() and ratePerHour() are abstract implicitly because they have no method bodies. calculateFee() is a default interface method, and validDuration() is a static interface method. Chargeable is an abstract class with state (ratePerUnit) and a constructor.

What cannot be expressed in an interface: ordinary per-object instance fields and constructors cannot be declared as interface state/constructors. Interface fields are implicitly public static final constants. A class can implement multiple interfaces, but it can extend only one class; therefore a class that extends Chargeable cannot also extend Car. That is the key trade-off in this reversed design. Use composition or separate interfaces if a class needs both car behaviour and charging.

For n fee calls, the default method takes O(1) time per call and O(n) total time. The abstract Chargeable object stores O(1) state per object. This design is possible but may be less convenient when vehicle subclasses need shared mutable state or class inheritance.